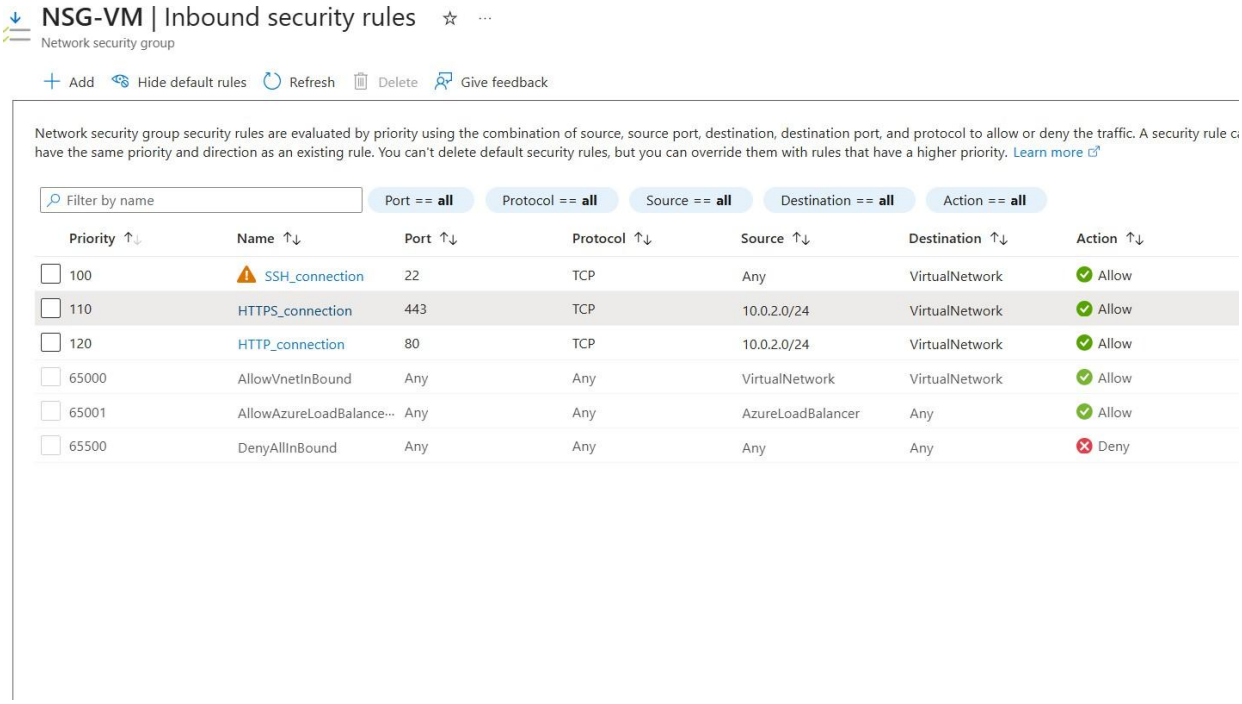
Architecture overview

One virtual network was created named –Project-VirtualNetwork.

Two subnets were created inside – VM subnet and Web Application Firewall subnet.

More on the WAF would be in the latter part of the report, but in summary it has Application gateway which receives HTTP and HTTPS requests, check them for potential command injections and then forwards to the VM subnet.

VM subnet is configured with Network Security Group rules.



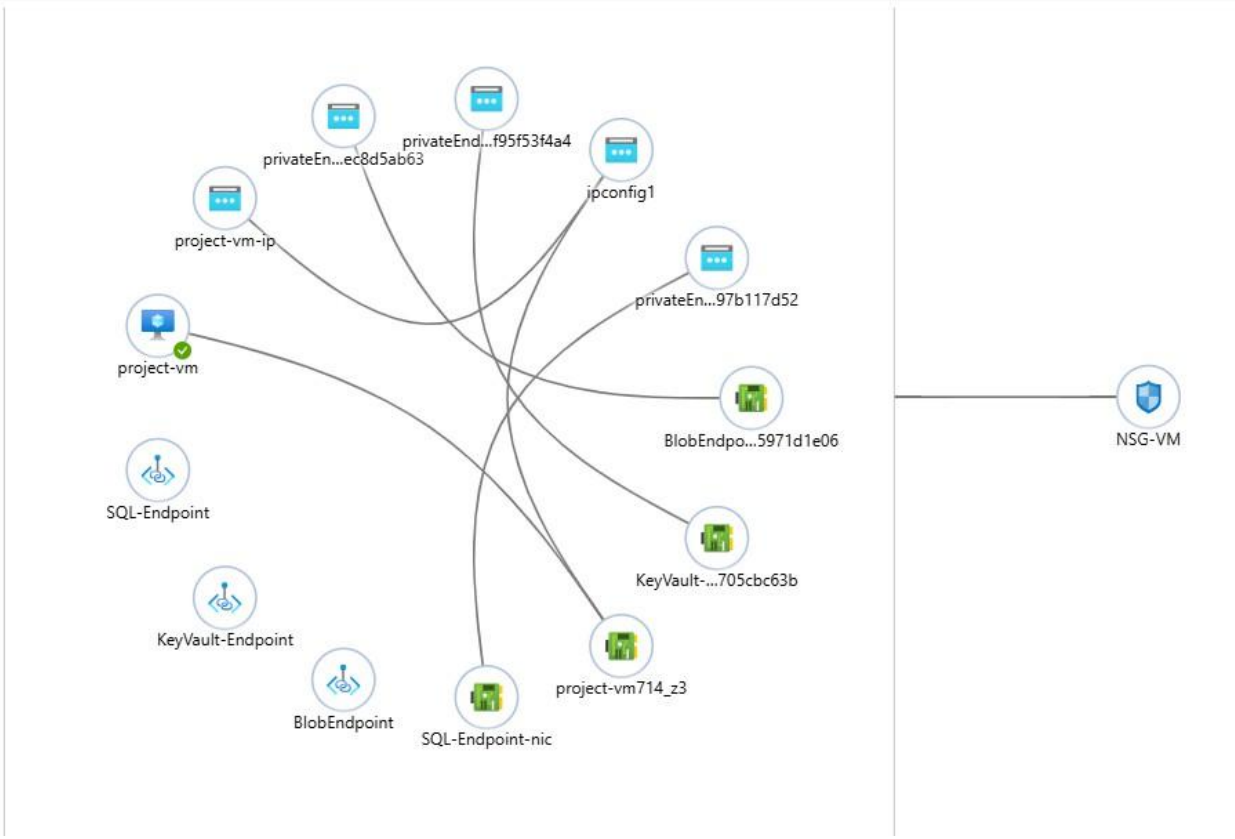
NSG Inbound rules are configured so that SSH connection is allowed from and source, but the HTTP and HTTPS connections are only accepted from the WAF subnet, so all the request will be checked by the firewall.

All other connection attempts would be denied.

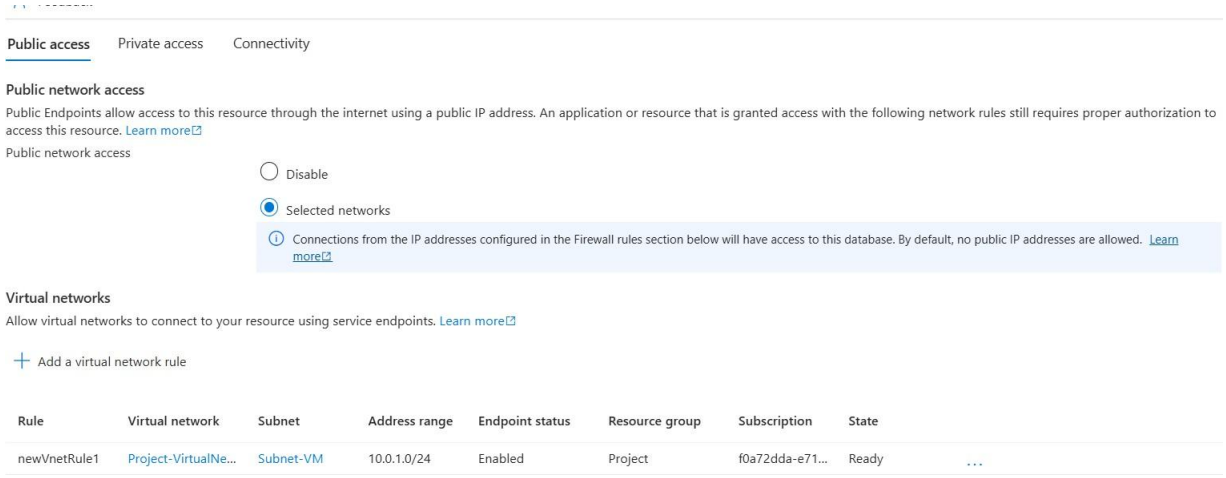
Second NSG was created for the WAF subnet which has Application Gateway in it. We only need HTTP and HTTPS connections to be allowed on this subnet, because those are the connections which will be than routed to the VM subnet, and all other connections would be useless or even harmful for the project. Also, the 65200-65535 port range was allowed as it was required by the Azure for Application Gateways.

Priority	Name	Port	Protocol	Source	Destination	Action
100	HTTP-WAF-ALLOW	80	TCP	Any	Any	Allow
110	HTTPS-WAF-ALLOW	443	TCP	Any	Any	Allow
120	WAF-AZURE-ALLOW	65200-65535	Any	Any	Any	Allow
65000	AllowVnetInBound	Any	Any	VirtualNetwork	VirtualNetwork	Allow
65001	AllowAzureLoadBalancerI···	Any	Any	AzureLoadBalancer	Any	Allow
65500	DenyAllInBound	Any	Any	Any	Any	Deny

It may be a security concern, but as the second layer of defense was configured as a NSG rules for the VM-subnet, and the UFW firewall rules would be configured on the VM itself, the potential risk coming from allowing those ports would be significantly less than the risk of command injection attacks, from which WAF protect the Project Web Server.



As a database for the project Azure SQL Database is used. To meet the requirement of Database being accessible only through the VM Public network access was enabled only for the VM subnet inside of the Project Virtual Network. With all other connections being disabled.



On the screenshot below it can be seen that connection to the database was only successful from the VM.

```
(venv) azureuser@project-vm:~$ sqlcmd -S project-sql-server-b001771480.database.windows.net -U b00177480 -P 'Sql_passwor
d!!#'
1>
```

Strong Authentication and Password Management

Password Authentication and Root Login were disabled on the VM by editing the sshd_config file, so users could only connect with their respective Private Key and could not connect as a root user.

Also, on creation VM was configured with being accessed only with SSH private key.

```
(venv) azureuser@project-vm:~$ sudo grep -E "PasswordAuthentication|PubkeyAuthentication" /etc/ssh/sshd_config
#PubkeyAuthentication yes
#PasswordAuthentication no
# PasswordAuthentication.  Depending on your PAM configuration,
# PAM authentication, then enable this but set PasswordAuthentication
(venv) azureuser@project-vm:~$
```

```
(venv) azureuser@project-vm:~$ sudo grep PermitRootLogin /etc/ssh/sshd_config
#PermitRootLogin prohibit-password
# the setting of "PermitRootLogin prohibit-password"
```

Even though the passwords are not used in this network configuration, PAM policies were still created for the potential future use. First of all the appropriate password requirements were configured by adding the following line to the common-password file:

password requisite pam_pwquality.so retry=3 minlen=12 difok=3 ucredit=-1 lcredit=-1 dcredit=-1 ocredit=-1

This line ensured that the password has minimal length of 12, need to have at least 1 uppercase and lowercase characters, digit and special symbol. Also, next password would be required to have at least three characters different from an old one.

```
(venv) azureuser@project-vm:~$ sudo grep pam_pwquality.so /etc/pam.d/common-password
password requisite pam_pwquality.so retry=3 minlen=12 difok=3 ucredit=-1 lcredit=-1 dcredit=-1 ocredit=-1
```

Also, the condition was added to the common-auth file so that after 5 unsuccessful attempts the user would be blocked for 15 minutes. It was done by adding the following line to the file:

auth required pam_tally2.so onerr=fail deny=5 unlock_time=900

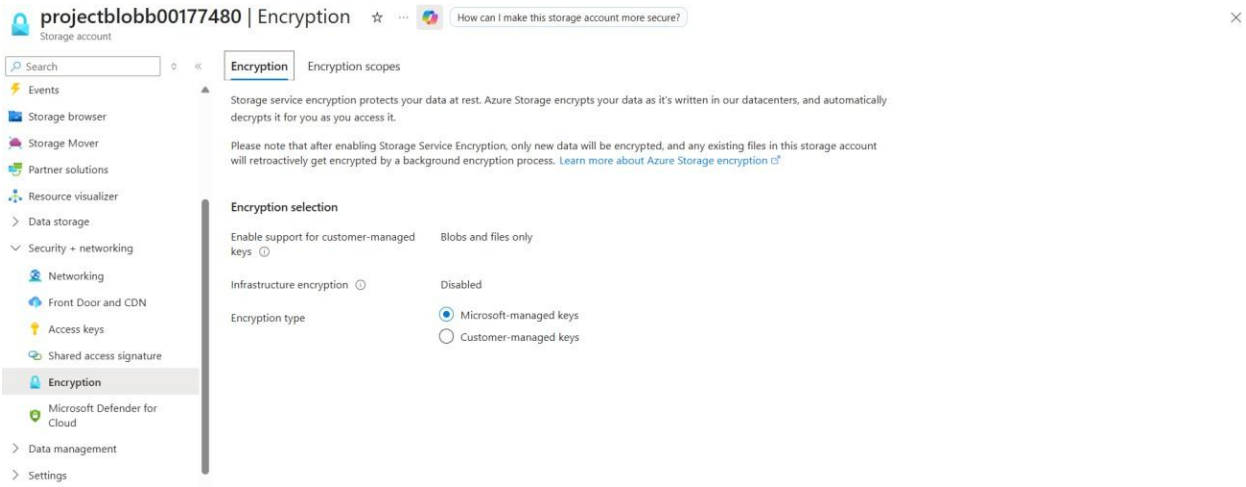
```
(venv) azureuser@project-vm:~$ sudo grep -E "faillock|tally" /etc/pam.d/*
/etc/pam.d/common-auth:auth required pam_tally2.so onerr=fail deny=5 unlock_time=900
```

Encryption in Transit and at Rest

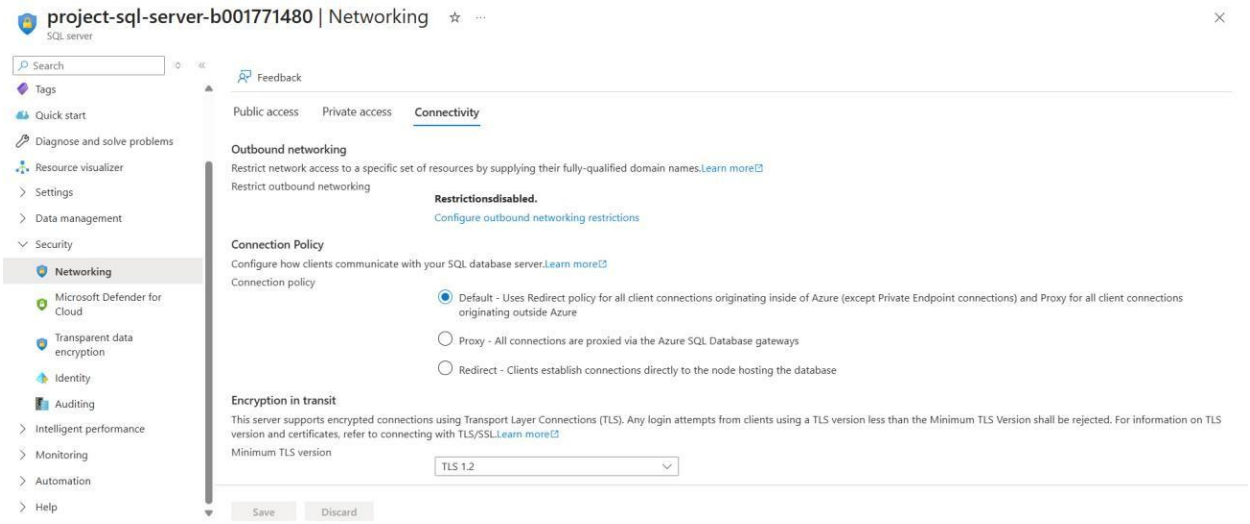
Transparent data encryption was enabled on SQL Database.



Encryption was enabled for StorageBlob with Microsoft Managed Keys.



SSL/TSL connections were enforced in Database settings so data would be transferred only in encrypted state from or to the user.



To enable the HTTPS connections, I generated a new self-signed certificate in PowerShell and then added then added new backend setting to my WAF-gateway where I enabled listening on port 443 and added my certificate there.

```
PS C:\Users\Roma_the_Bombat_User> $cert = New-SelfSignedCertificate -Subject "CN=$certname" -CertStoreLocation "Cert:\CurrentUser\My" -KeyExportPolicy Expor
table -KeySpec Signature -KeyLength 2048 -KeyAlgorithm RSA -HashAlgorithm SHA256
PS C:\Users\Roma_the_Bombat_User> Export-Certificate -Cert $cert -FilePath "C:\Users\Roma_the_Bombat_User\Desktop\$certname.cer"
```

Next the .pfx certificate was created using the following commands:

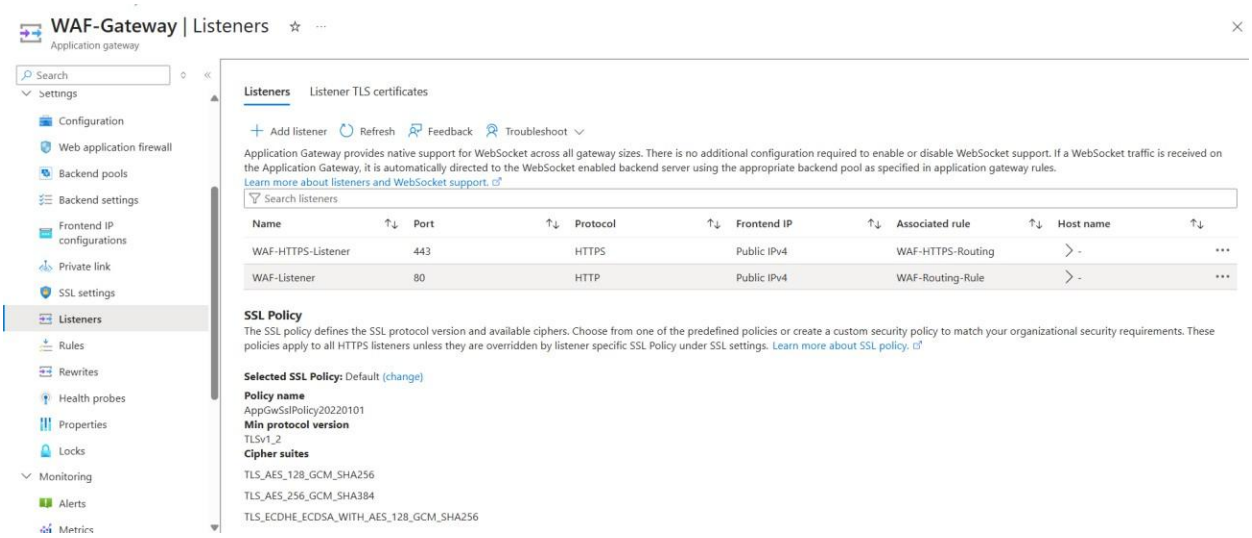
Created a private key: *$ openssl genrsa -out myapp.key 2048*

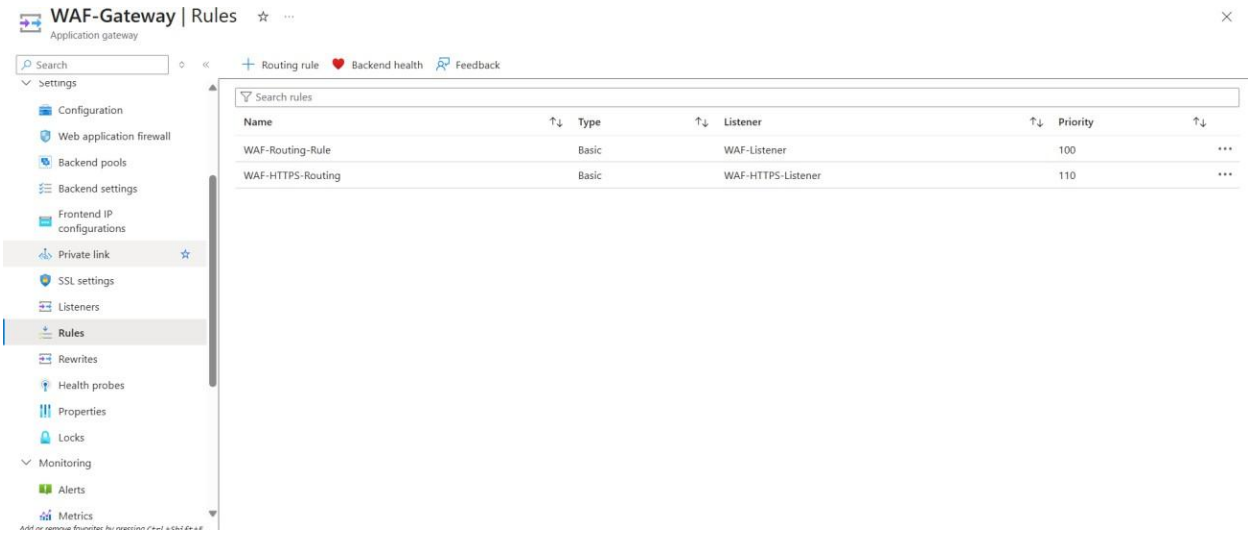
Created a certificate signing request : *$ openssl req -new -key myapp.key -out myapp.csr*

Generated the self-signed certificate: *openssl x509 -req -days 365 -in myapp.csr -signkey myapp.key -out myapp.crt*

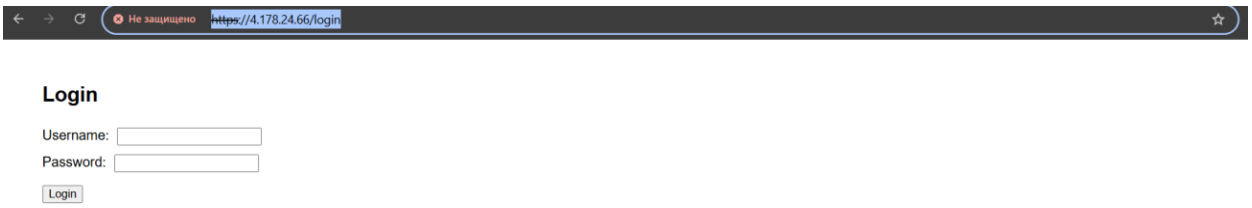
Exported it as .pfx file as it is required by Azure: *openssl pkcs12 -export -out myapp.pfx -inkey myapp.key -in myapp.crt.*

With this certificate a new listener was created in my WAF gateway, and then added a new routing rule for the HTTPS protocol.





Now the web server was accessible not only with the HTTP protocol, but HTTPS as well.



It can be seen that the https protocol is crossed out because of the certificate used, which is not signed by any trusted authority. But the main point is that HTTPS was enabled for web traffic.

System Hardening and Monitoring

UFW was configured with the same setting as used for the NSG.

Connection was only allowed for SSH on port 22, HTTP on port 80, and HTTPS on port 443 for Ipv4 and Ipv6.

```
^C(venv) azureuser@project-vm:~/project-webapp$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To                         Action      From
--                         ------      ----
22/tcp                     ALLOW IN    Anywhere
80/tcp                     ALLOW IN    Anywhere
443/tcp                    ALLOW IN    Anywhere
22/tcp (v6)                ALLOW IN    Anywhere (v6)
80/tcp (v6)                ALLOW IN    Anywhere (v6)
443/tcp (v6)               ALLOW IN    Anywhere (v6)
```

Attempt to ping public address of the VM was not successful, but it is due to the presence of the NSG rules which were not configured to allow this type of connection. However, if there would be the second VM in the same Virtual Network, UFW rules would be used to enforce the same level of security inside the Virtual Network.

```
C:\Users\Roma_the_Bombat_User\Downloads>ping 172.189.57.237

Pinging 172.189.57.237 with 32 bytes of data:
Request timed out.
```

To prevent SSH brute force attacks SSH password authentication and Root Login were disabled as it was shown in the previous section. But to add security fail2ban service was deployed on the VM. This service checks the logs in real time on the system and if too many SSH connection attempts from certain IP are unsuccessful – the IP will be blocked for a time period determined by the configuration.

Following group of command were used to install and enable fail2ban system:

sudo apt update

sudo apt install fail2ban -y

sudo systemctl enable fail2ban

sudo systemctl start fail2ban

After running the command to check the fail2ban status on the next day, the following report was given.

```
(venv) azureuser@project-vm:~/project-webapp$ sudo fail2ban-client status sshd
Status for the jail: sshd
|- Filter
|  |- Currently failed: 2
|  |- Total failed:     807
|  `- Journal matches:  _SYSTEMD_UNIT=sshd.service + _COMM=sshd
`- Actions
   |- Currently banned: 1
   |- Total banned:     82
   `- Banned IP list:   188.166.215.51
```

In one day of work fail2ban managed to ban 82 IP addresses so no additional tests would even be needed to run. Such result only proves that deployment of this service was a necessity under the condition where SSH connection is open to the Internet.

Next, the unattended-upgrades system was installed on the VM. It is necessary that the OS is always up to date in terms of security, as many of the cybersecurity attacks would be prevented if the necessary security updates would be installed on the systems in advance.

The system was installed with the following command:

sudo apt install unattended-upgrades –y

After that, configuration file was checked for the following:

```
(venv) azureuser@project-vm:~/project-webapp$ cat /etc/apt/apt.conf.d/20auto-upgrades
APT::Periodic::Update-Package-Lists "1";
APT::Periodic::Unattended-Upgrade "1";
```

Update-Package-Lists "1" means that the package index is updated daily

Unattended-Upgrade "1" means that security updates are installed automatically

```
Unattended-Upgrade::Allowed-Origins {
        "${distro_id}:${distro_codename}";
        "${distro_id}:${distro_codename}-security";
        // Extended Security Maintenance; doesn't necessarily exist for
        // every release and this system may not have it installed, but if
        // available, the policy for updates is such that unattended-upgrades
        // should also install from here by default.
        "${distro_id}ESMApps:${distro_codename}-apps-security";
        "${distro_id}ESM:${distro_codename}-infra-security";
//      "${distro_id}:${distro_codename}-updates";
//      "${distro_id}:${distro_codename}-proposed";
//      "${distro_id}:${distro_codename}-backports";
};
```

Then the whole file was opened with nano to check if the line that allows security updates in uncommented. If needed, last three lines could also be uncommented.

After enabling this service with the following block of code:

sudo systemctl enable unattended-upgrades

sudo systemctl start unattended-upgrades

It is now possible to check the status and the log file of the system to check if it works as intended.

```
(venv) azureuser@project-vm:~/project-webapp$ sudo systemctl status unattended-upgrades
● unattended-upgrades.service - Unattended Upgrades Shutdown
     Loaded: loaded (/usr/lib/systemd/system/unattended-upgrades.service; enabled; preset: enabled)
     Active: active (running) since Mon 2025-12-08 23:20:19 UTC; 1 day 18h ago
       Docs: man:unattended-upgrade(8)
   Main PID: 759 (unattended-upgr)
      Tasks: 2 (limit: 9444)
     Memory: 12.1M (peak: 12.3M)
        CPU: 84ms
     CGroup: /system.slice/unattended-upgrades.service
             └─759 /usr/bin/python3 /usr/share/unattended-upgrades/unattended-upgrade-shutdown --wait-for-signal

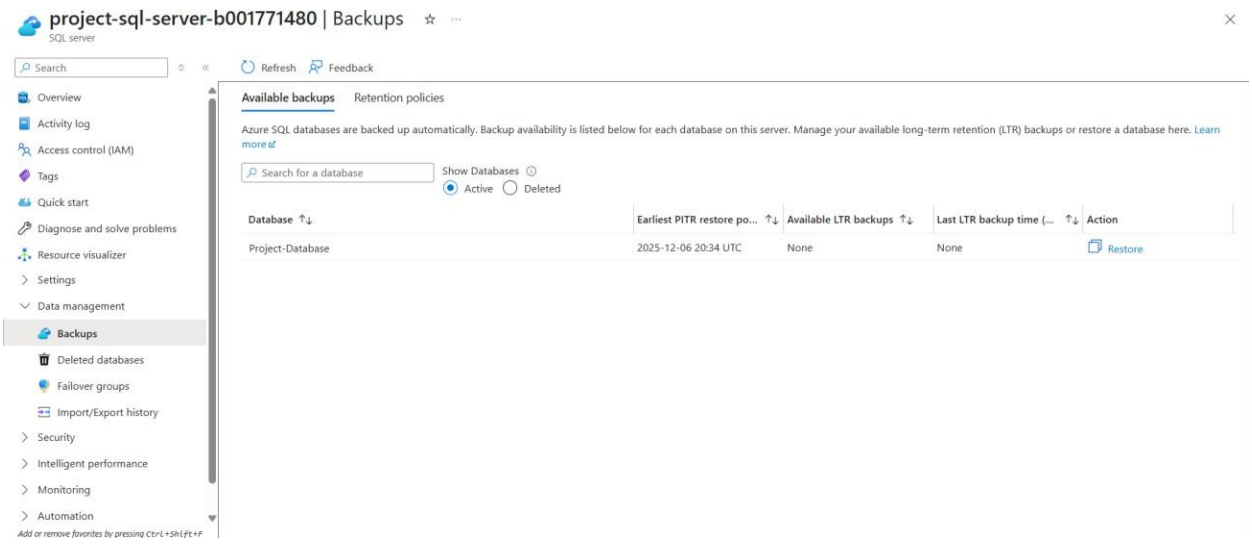
Dec 08 23:20:19 project-vm systemd[1]: Started unattended-upgrades.service - Unattended Upgrades Shutdown.
```

```
(venv) azureuser@project-vm:~/project-webapp$ cat /var/log/unattended-upgrades/unattended-upgrades.log
2025-12-08 16:55:27,183 INFO Starting unattended upgrades script
2025-12-08 16:55:27,184 INFO Allowed origins are: o=Ubuntu,a=noble, o=Ubuntu,a=noble-security, o=UbuntuESMApps,a=noble-a
pps-security, o=UbuntuESM,a=noble-infra-security
2025-12-08 16:55:27,184 INFO Initial blacklist:
2025-12-08 16:55:27,184 INFO Initial whitelist (not strict):
2025-12-08 16:55:31,571 INFO Packages that will be upgraded: bind9-dnsutils bind9-host bind9-libs libpython3-stdlib libp
ython3.12-minimal libpython3.12-stdlib libpython3.12t64 libssh-4 libxml2 linux-cloud-tools-common linux-tools-common pyt
hon3 python3-minimal python3.12 python3.12-minimal
2025-12-08 16:55:31,572 INFO Writing dpkg log to /var/log/unattended-upgrades/unattended-upgrades-dpkg.log
2025-12-08 16:55:59,132 INFO All upgrades installed
2025-12-08 18:52:56,665 INFO Starting unattended upgrades script
2025-12-08 18:52:56,665 INFO Allowed origins are: o=Ubuntu,a=noble, o=Ubuntu,a=noble-security, o=UbuntuESMApps,a=noble-a
pps-security, o=UbuntuESM,a=noble-infra-security
2025-12-08 18:52:56,665 INFO Initial blacklist:
```

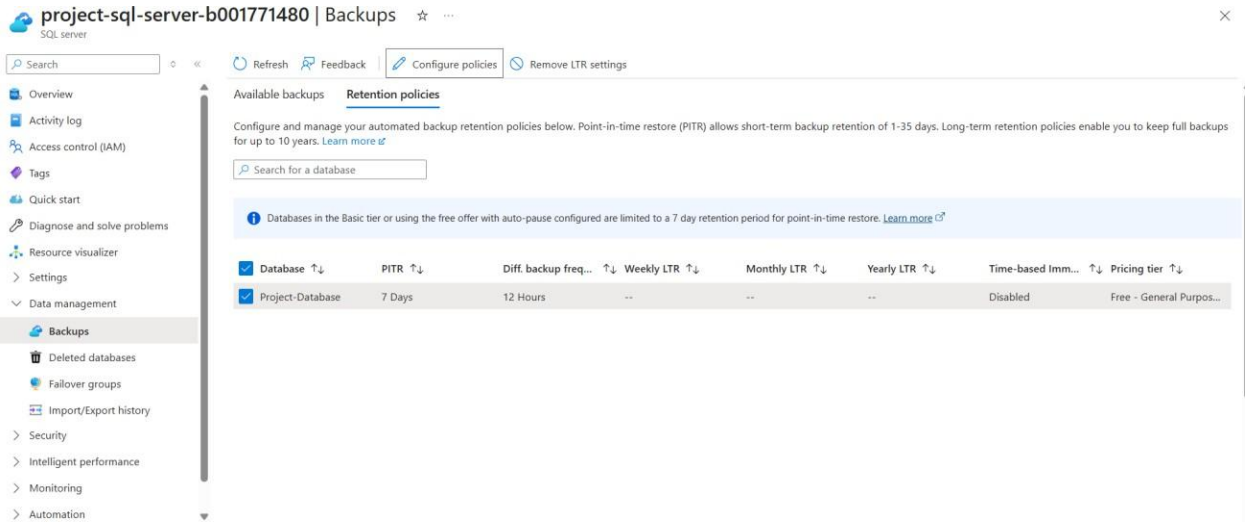
It can be seen from the screenshots that the unattended-upgrades service is active and already performed some updates on the system.

Logging, Backup and Audit Trail

Backups on SQL server were enabled in Azure:



Retention policy was configured so that point in time restore backup would be kept for 7 days before deletion.



To create the OS level backup for web server, following script was created:

#!/bin/bash

TIMESTAMP=$(date +"%Y%m%d_%H%M%S")

WEBAPP_DIR="/home/azureuser/project-webapp"

BACKUP_DIR="/backups/project-webapp"

BACKUP_FILE="$BACKUP_DIR/webapp_backup_$TIMESTAMP.tar.gz"

tar -czf $BACKUP_FILE $WEBAPP_DIR

echo "Backup created at $TIMESTAMP: $BACKUP_FILE" >> $BACKUP_DIR/webapp_backup.log

#!/bin/bash

TIMESTAMP variable is used to create a timestamp for the future backup file, so it would be easier to differentiate them.

WEBAPP_DIR variable is storing the path to directory where the web application is stored.

BACKUP_DIR variable that stores path to the directory where backups files will be created.

BACKUP_FILE this variable creates a template for how the file will be named and what format it will be stored in. TIMESTAMP will make each file have different name.

tar -czf $BACKUP_FILE $WEBAPP_DIR

tar command will create a compressed archive of the application directory, -c options specifies that new archive will be created, -z means that gzip will be used to compress the archive, and -f + $BACKUP_FILE means that

BACKUP_FILE variable will be used to name the archive and $WEBAPP_DIR variable points to the directory to be archived.

echo "Backup created at $TIMESTAMP: $BACKUP_FILE" >> $BACKUP_DIR/webapp_backup.log

Last line will goes to log each backup creation to the designated log file.



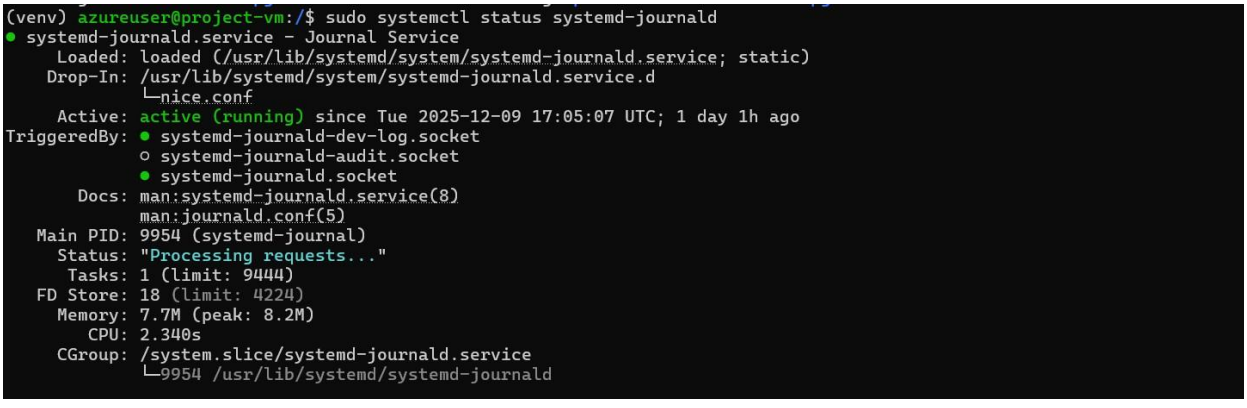
It can be seen how three backup files were created and one log file exist.



Log file has record of creation for each of the archives.

To configure system logging both the journald and rsyslog were enabled on the vm.





In the journald configuration file changes were applied so that the storage of logs would be persistent and kept for 1 month. Also, the option was enabled to forward logs to the rsyslog as it would be more convenient to access all logs with a single service.

```
#Storage=persistent
#Compress=yes
#Seal=yes
#SplitMode=uid
#SyncIntervalSec=5m
#RateLimitIntervalSec=30s
#RateLimitBurst=10000
#SystemMaxUse=200M
#SystemKeepFree=50M
#SystemMaxFileSize=
#SystemMaxFiles=100
#RuntimeMaxUse=
#RuntimeKeepFree=
#RuntimeMaxFileSize=
#RuntimeMaxFiles=100
#MaxRetentionSec=1month
#MaxFileSec=1month
#ForwardToSyslog=yes
#ForwardToKMsg=no
#ForwardToConsole=no
#ForwardToWall=yes
#TTYPath=/dev/console
#MaxLevelStore=debug
#MaxLevelSyslog=debug
#MaxLevelKMsg=notice
#MaxLevelConsole=info
```

For rsyslog separate file was created to store logs connected to sshd. As it many of the security operations performed were connected to this programs.

```
  GNU nano 7.2                                  /etc/rsyslog.d/50-sshd.conf
if ($programname == "sshd") then /var/log/sshd.log
& stop




                                        [ Read 2 lines ]
```

```
(venv) azureuser@project-vm:~$ tail /var/log/sshd.log
2025-12-10T19:17:07.451546+00:00 project-vm sshd[23119]: PAM adding faulty module: pam_pwquality.so
2025-12-10T19:17:07.480256+00:00 project-vm sshd[23119]: Received disconnect from 193.46.255.33 port 56326:11:  [preauth]
2025-12-10T19:17:07.480344+00:00 project-vm sshd[23119]: Disconnected from authenticating user root 193.46.255.33 port 56326 [preauth]
2025-12-10T19:21:27.297088+00:00 project-vm sshd[23166]: Invalid user user from 2.57.121.25 port 42881
2025-12-10T19:21:27.297909+00:00 project-vm sshd[23166]: PAM unable to dlopen(pam_tally2.so): /usr/lib/security/pam_tally2.so: cannot open shared object file: No such file or directory
2025-12-10T19:21:27.298022+00:00 project-vm sshd[23166]: PAM adding faulty module: pam_tally2.so
2025-12-10T19:21:27.299130+00:00 project-vm sshd[23166]: PAM unable to dlopen(pam_pwquality.so): /usr/lib/security/pam_pwquality.so: cannot open shared object file: No such file or directory
2025-12-10T19:21:27.299224+00:00 project-vm sshd[23166]: PAM adding faulty module: pam_pwquality.so
2025-12-10T19:21:27.473412+00:00 project-vm sshd[23166]: Received disconnect from 2.57.121.25 port 42881:11: Bye [preauth]
2025-12-10T19:21:27.473558+00:00 project-vm sshd[23166]: Disconnected from invalid user user 2.57.121.25 port 42881 [preauth]
```

It can be seen how the sshd related operations are now logged into the new log file.

Log rotation was also enabled on the VM with the logrotate service.

Log rotation was enabled for syslog files, and new file created for sshd. Rotation would be performed weekly for al files, and 4 old version of the files would be kept.





Bash Script

For the script it was decided to check if the services configured on the VM.

Script works the following way:

if systemctl is-active --quiet rsyslog; then

echo "rsyslog is running"

else

echo "rsyslog is NOT running"

fi

This script checks the process with is-active method, then block the output of the command, but if the command succeeds it echoes that the process is running, otherwise it tells that the service is not running.

SCRIPT_PATH="/home/azureuser/backup_webapp.sh"

if crontab -l | grep -q "$SCRIPT_PATH"; then

echo "Cron job for $SCRIPT_PATH is scheduled."

else

echo "Cron job for $SCRIPT_PATH is NOT scheduled."

fi

This part of the script checks if the custom script created to make backups of the project directory is scheduled in cron. If command that greps the path to the script is successful – it will echo about it to the user, otherwise it echoes that the script is not scheduled.

```
azureuser@project-vm:~$ ./test_script.sh
Fail2ban is running
rsyslog is running
auditd is running
journald is running
UFW is running and active
logrotate is NOT running
unattended-upgrades is running
Cron job for /home/azureuser/backup_webapp.sh is scheduled.
azureuser@project-vm:~$ systemctl status logrotate
○ logrotate.service - Rotate log files
     Loaded: loaded (/usr/lib/systemd/system/logrotate.service; static)
     Active: inactive (dead) since Fri 2025-12-12 00:00:03 UTC; 2h 16min ago
TriggeredBy: ● logrotate.timer
       Docs: man:logrotate(8)
             man:logrotate.conf(5)
    Process: 31058 ExecStart=/usr/sbin/logrotate /etc/logrotate.conf (code=exited, status=0/SUCCESS)
   Main PID: 31058 (code=exited, status=0/SUCCESS)
        CPU: 84ms

Dec 12 00:00:03 project-vm systemd[1]: Starting logrotate.service - Rotate log files...
```

Here, logrotate was also added to check if script also finds services that are not active, which works.

Logrotate should not be active all the time as it only rotates logs on timer, and it can be seen how the last rotation was completed recently.

Users

Following groups were created for the project:

QA Testers, Developers, Managers with 2 users in each group.

```
azureuser:x:1000:1000:Ubuntu:/home/azureuser:/bin/bash
dev1:x:1001:1004::/home/dev1:/bin/sh
dev2:x:1002:1005::/home/dev2:/bin/sh
qa1:x:1003:1006::/home/qa1:/bin/sh
qa2:x:1004:1007::/home/qa2:/bin/sh
pm1:x:1005:1008::/home/pm1:/bin/sh
pm2:x:1006:1009::/home/pm2:/bin/sh
```

SSH keys were generated for each user to be able to SSH with their respective key to the machine.

The directory where the web application project was stored exists on the VM so each group on the VM should have different level of access to this directory.

sudo setfacl -R -m g:qatesters:rx project-webapp

sudo setfacl -R -m g:projectmanagers:r project-webapp

sudo setfacl -R -m g:developers:rwx project-webapp

QA Testers group now has the right to read and execute code, but not change it, it is necessary so they would be able to actually test the code of the project.

Project managers group only got the rights to read contents of the directory as it is usually not their job to run code or change it themselves.

Finally, the developers group has rights to read right and execute code in the directory to actually be able to work on the project.

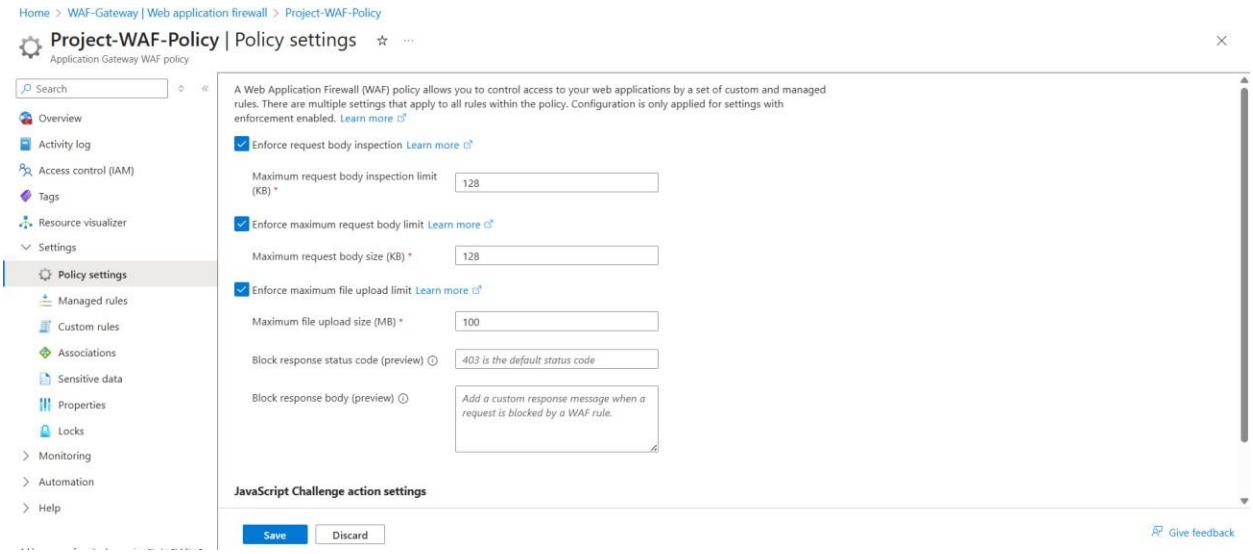
```
azureuser@project-vm:~$ getfacl /home/azureuser/project-webapp
getfacl: Removing leading '/' from absolute path names
# file: home/azureuser/project-webapp
# owner: azureuser
# group: azureuser
user::rwx
group::---
group:developers:rwx
group:qatesters:r-x
group:projectmanagers:r--
mask::rwx
other::---
```

Additional Security Implementations

WAF was already partially covered in this report. New application gateway was created before our VM which is listening on ports 80 and 443 for HTTP and HTTPS request, and then forwards them to the VM subnet. Firewall

there check request for potential command injection attacks. Rules for the firewall are provided by Azure and have all the OWASP 3.2 firewall ruleset.

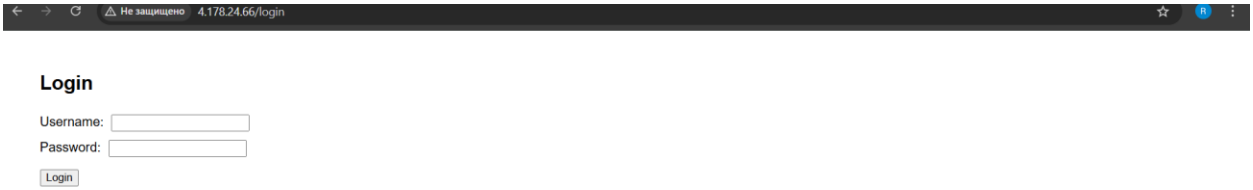
If the command injection attack occurs, firewall is blocking the 403 status code.



To test if the firewall works for both of our protocols following links were used: http://4.178.24.66/?id=1%27 and https://4.178.24.66/?id=1%27.

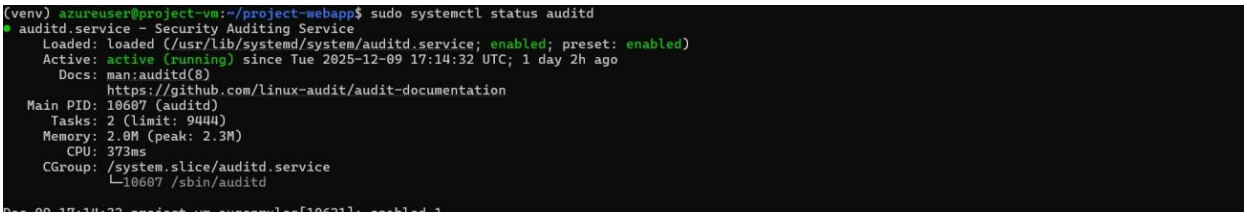


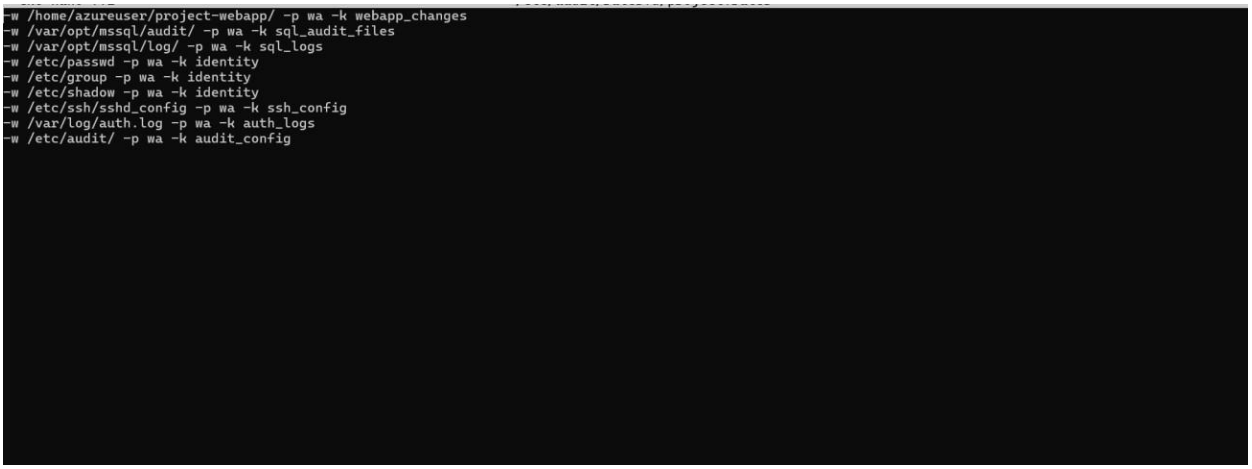




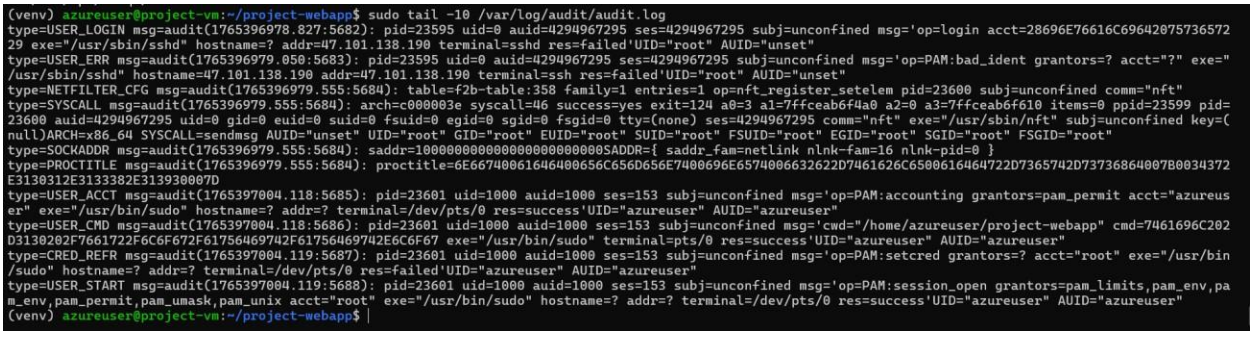
It can be seen how the WAF would block request with potentially malicious or unexpected content or symbols, but allows normal requests.

Auditd was enabled on the VM and configured with the rules seen on the second screenshot below:





-w specifies directory or file to monitor and –p means permissions which will be monitored such as read, write, execute, attribute change –k provides identifier for each rule so that it will be easier to find it in the logs.



Auditd log file already has different entries about actions performed in the monitored files and directories.

Auditd would also provide file integrity as it will monitor and log when files within the scope of its rules are changed.

Another possible security implementation to provide file integrity would be file integrity monitoring in the Microsoft Defender for cloud, but different subscription would be required for it.